

SPARKY

Full Project Specification & Standards Sheet

Sports Betting Prediction & Parlay Intelligence App

Version 1.0

May 2026

Prepared for: Harrison Eller
Single Source of Truth Document

Table of Contents

1. Project Overview
2. Tech Stack & Architecture
3. Data Models / Database Schema
4. Intelligence Layer (SOW 1)
5. UI/UX Standards & Features
6. Coding Standards
7. Development Workflow
8. Milestones & Acceptance Criteria
9. Future Expansion
10. How to Use This Document

1. Project Overview

Name & Branding

Name: Sparky

Tagline: Sharp NFL Predictions • Intelligent Parlays • Real Edge

Core Purpose

Build a professional sports betting prediction & parlay intelligence web app (PWA-ready) focused on NFL as the first sport. The app provides individual game predictions, confidence scores, market signal tags, ranked three-leg parlays, and historical accuracy reporting.

POC Goal

Prove the complete end-to-end pipeline for NFL: odds ingestion → snapshots → signals → predictions → 3-leg parlay ranking → historical accuracy dashboard.

Key Principles (from SOW)

- One sport first (NFL)
- Universal engine + sport-specific modules
- Rule-based core + ML-enhanced confidence & ranking
- Strong, thoughtful intelligence (not overly complex)
- Sleek, modern, professional “sexy” UI

2. Tech Stack & Architecture

Core Technology Stack

- **Frontend / Full-Stack:** Next.js 15 (App Router) + TypeScript
- **Styling:** Tailwind CSS + shadcn/ui + Framer Motion
- **Database:** Supabase (PostgreSQL)
- **State Management:** TanStack Query (React Query)
- **HTTP Client:** Axios
- **Charts:** Recharts
- **PWA:** Built-in Next.js PWA support
- **Odds Source:** The Odds API (primary)
- **ML:** XGBoost / scikit-learn (via Python or Edge functions)
- **Deployment:** Vercel + Supabase

Architecture Layers

1. **Ingestion Layer:** The Odds API → Raw Snapshots
2. **Normalization Layer:** Clean team-level odds + T1/T2/T3 snapshots
3. **Intelligence Layer:** Universal + NFL-specific (lib/quant/)
4. **Prediction & Parlay Engine:** Confidence scoring + 3-leg ranking
5. **Storage Layer:** Supabase PostgreSQL
6. **API Layer:** Next.js Route Handlers
7. **Presentation Layer:** Responsive Dashboard + PWA

3. Data Models / Database Schema (Supabase)

Core Tables

raw_odds_snapshots

Purpose: Store exact API responses and source metadata.

Fields: id (uuid), sport, league, snapshot_label (T1/T2/T3), raw_payload (jsonb), fetched_at

normalized_odds

Purpose: Store cleaned team-level odds used by the model.

Fields: id, game_id, home_team, away_team, commence_time, book, home_moneyline, away_moneyline, decimal_odds, implied_prob, vig_adjusted_prob, is_favorite, snapshot_label, timestamp

game_predictions

Purpose: Store individual game predictions and signals.

Fields: game_id (PK), predicted_winner, confidence_score (0-100), signals (text[]), explanation, created_at

parlay_rankings

Purpose: Store all ranked three-leg parlay outputs.

Fields: slate_id, rank, leg1/leg2/leg3_game_id, parlay_odds_american, parlay_odds_decimal, implied_prob, underdog_count, confidence_score, signal_alignment, explanation

historical_results

Purpose: Store outcomes for accuracy tracking.

Fields: game_id, actual_winner, prediction_correct, parlay_rank, sport, settled_at

4. Intelligence Layer (SOW 1 — Core of Sparky)

Universal Engine

- American ↔ Decimal odds conversion
- Implied probability + vig adjustment
- T1/T2/T3 movement deltas
- Favorite / Underdog classification
- Signal detection framework

Required Signals (with definitions)

- **Steam Move:** Heavy line movement toward one side
- **Resistance:** Line not moving despite public money
- **Anchor Favorite:** Strong favorite with multi-book support
- **Underdog Strengthening:** Dog money coming in
- **Market Compression:** Lines tightening
- **Reverse Line Movement, Trap Risk, Coin Flip, High-Variance Game**
- **Multi-book Confirmation / Disagreement**
- **Late Movement, False Stability**

Confidence Scoring

Rule-based base score (45-95) + feature-engineered boosts from movement magnitude, consensus, home advantage, signal strength, etc.

NFL-Specific Module

- Key player heuristics (QB, WR, etc.)
- Injury stub (expandable via ESPN API)
- Rest / short week considerations

Parlay Ranking (3-leg)

Generate all 8 combinations. Rank by composite score: confidence × signal alignment × underdog balance × implied edge. Not just raw odds.

Historical Accuracy Metrics

- Individual pick accuracy
- Rank #1 hit rate
- Top-3 / Top-4 containment rate
- Rolling 7/21/30-day windows

5. UI/UX Standards & Features (SOW 2)

Design Language

- Dark theme (zinc-950 base)
- Premium betting-app aesthetic: glassmorphism, emerald-cyan gradients
- Smooth animations (Framer Motion)
- Professional, clean, sexy, modern
- Responsive + PWA (installable on mobile)

Core Views

8. **Dashboard:** Today's slate with prediction cards
9. **Game Detail:** Odds snapshots, movement charts, full signal breakdown
10. **Parlay Builder:** Select 3 games → live ranked 8 outcomes
11. **Historical Accuracy:** Rolling windows, charts, by signal type
12. **Admin/Debug:** Last pull status, raw data viewer, pipeline health

Prediction Card Elements

- Teams + current odds
- Predicted winner + confidence %
- Signal tags (colored pills)
- Plain-English explanation
- "Add to Parlay" button

6. Coding Standards

- **Language:** TypeScript (strict mode)
- **Components:** Functional components with hooks
- **Server Components:** Preferred where possible (Next.js 15)
- **Folder Structure:**

app/ — Pages & layouts

lib/quant/ — Intelligence engine

lib/supabase/ — Client & queries

components/ — Reusable UI

- **Naming:** camelCase for JS/TS, kebab-case for files
- **Error Handling:** Comprehensive + logging
- **Secrets:** Never hardcode — use .env.local
- **Comments:** Required on complex quant logic

7. Development Workflow

Local Development Commands

Initial Setup:

```
npx create-next-app@latest . --typescript --tailwind --eslint --app --yes
```

```
npm install @supabase/supabase-js @tanstack/react-query axios date-fns framer-motion lucide-react recharts
```

```
cp .env.example .env.local
```

Run: `npm run dev`

Git & Collaboration

- Feature branches → Pull Requests
- Iterative delivery (as demonstrated)

8. Milestones & Acceptance Criteria

Phase 1 (Completed)

Foundation, basic dashboard, quant engine, The Odds API integration, PWA-ready.

Phase 2

Full parlay builder + ranking, Supabase persistence, advanced signals.

Phase 3

Historical accuracy + dashboards, ML training pipeline.

Phase 4

Game details, polish, full PWA, admin tools.

Phase 5

Production deployment + multi-user readiness.

Final Acceptance Criteria (SOW-aligned)

- All required signals and calculations working
- 3-leg parlay ranking complete
- Historical accuracy metrics displayed
- Clean, professional UI
- Extensible architecture

9. Future Expansion

- Add more sports via modular imports
- User accounts & saved parlays
- Subscription tiers
- Advanced ML (player props, etc.)
- AWS migration path (Lambda + DynamoDB) if needed

10. How to Use This Document

This is the **Single Source of Truth** for the Sparky project.

To continue development or restart prompting:

Copy and paste this entire document (or relevant sections) into a new conversation and say:

"Continue building Sparky from the full spec. Next priority: [your request]"

You now have a complete, self-contained blueprint for the entire project.

— End of Document —